BAIL security

Lista Dao
Credit Liquidation

FINAL REPORT

February '2026

Disclaimer:

Security assessment projects are time-boxed and often reliant on information that may be provided by a client, its affiliates, or its partners. As a result, the findings documented in this report should not be considered a comprehensive list of security issues, flaws, or defects in the target system or codebase.

The content of this assessment is not an investment. The information provided in this report is for general informational purposes only and is not intended as investment, legal, financial, regulatory, or tax advice. The report is based on a limited review of the materials and documentation provided at the time of the audit, and the audit results may not be complete or identify all possible vulnerabilities or issues. The audit is provided on an "as-is," "where-is," and "as-available" basis, and the use of blockchain technology is subject to unknown risks and flaws.

The audit does not constitute an endorsement of any particular project or team, and we make no warranties, expressed or implied, regarding the accuracy, reliability, completeness, or availability of the report, its content, or any associated services or products. We disclaim all warranties, including the implied warranties of merchantability, fitness for a particular purpose, and non-infringement.

We assume no responsibility for any product or service advertised or offered by a third party through the report, any open-source or third-party software, code, libraries, materials, or information linked to, called by, referenced by, or accessible through the report, its content, and the related services and products. We will not be liable for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract.

The contract owner is responsible for making their own decisions based on the audit report and should seek additional professional advice if needed. The audit firm or individual assumes no liability for any loss or damages incurred as a result of the use or reliance on the audit report or the smart contract. The contract owner agrees to indemnify and hold harmless the audit firm or individual from any and all claims, damages, expenses, or liabilities arising from the use or reliance on the audit report or the smart contract.

By engaging in a smart contract audit, the contract owner acknowledges and agrees to the terms of this disclaimer.

1. Project Details

Important:
Please ensure that the deployed contract matches the source-code of the last commit hash.

Project	Lista Dao - Credit Liquidation - Audit Report
Website	lista.org
Language	Solidity
Methods	Manual Analysis
Github repository	https://github.com/lista-dao/moolah/tree/11efbec87a0156e867696d09793056665cdc34ad/src
Resolution 1	https://github.com/lista-dao/moolah/pull/137/changes/3be5a444d181f2a474be9dc8bded224da6f3c4ee

2. Detection Overview

Severity	Found	Resolved	Partially Resolved	Acknowledged (no changes made)	Failed resolution	Open
High	1	1				
Medium	2			2		
Low	2			2		
Informational	7			7		
Governance						
Total	12	1		11		

2.1 Detection Definitions

Severity	Description
High	The problem poses a significant threat to the confidentiality of a considerable number of users' sensitive data. It also has the potential to cause severe damage to the client's reputation or result in substantial financial losses for both the client and the affected users.
Medium	While medium level vulnerabilities may not be easy to exploit, they can still have a major impact on the execution of a smart contract. For instance, they may allow public access to critical functions, which could lead to serious consequences.
Low	Poses a very low-level risk to the project or users. Nevertheless the issue should be fixed immediately
Informational	Effects are small and do not post an immediate danger to the project or users
Governance	Governance privileges which can directly result in a loss of funds or other potential undesired behavior

3. Detection

MoolahOperateLib

The MoolahOperateLib is a simple library which is used by the Moolah contract in an effort to reduce contract size. It exposes similar logic as been before within the Moolah contract directly.

Privileged Functions

- none

Issue_01	Rounding concerns when repaying positions
Severity	**Informational**
Description	Within MoolahOperateLib::repayToMoolah, the asset amount specified by the user is converted to shares before being passed to Moolah::repay: *uint256 amountShares = amount.toSharesDown(market.totalBorrowAssets, market.totalBorrowShares);* *// using `shares` to ensure full repayment* *(assetsRepaid, sharesRepaid) = IMoolah(moolah).repay(* *_getMarketParams(moolah, marketId),* *0,* *amountShares,* *onBehalf,* *"",* *);* The conversion uses toSharesDown, which rounds down. As a result, a small number of "dust" shares may remain unpaid after repayment. If a broker position is closed while some borrowed shares remain due to rounding, the user may be unable to fully repay the final portion of their debt. While we did not identify any instance of this issue due to the exchange rate currently being 1, it may not pose a problem under the present design. However, if the exchange rate were to deviate, this issue could cause significant problems for users.
Recommendations	Consider acknowledging this issue. But make sure that the exchange rate between asset-share conversion doesn't change.
Comments / Resolution	Acknowledged

CreditBroker - Differential

The Moolah contract undergoes a differential audit based on the following diff:

https://www.diffchecker.com/4urEVOjq/

The following changes have been made:

- Decimal assumption explicitly set to 18
- Refactoring of the _borrow function
- Extension of the FixedLoanPositions struct
- Refactoring of the _repay function
- Outsourcing of:
    - _borrowFromMoolah
    - _repayToMoolah
    - _supplyToMoolahVault

Appendix: Bad Debt

A bad debt mechanism has been introduced which is triggered whenever a position gets liquidated. the bad debt will be written off to Moolah suppliers and position owners will get their debt cleared.

Appendix: Liquidation

Whenever a position has been expired, it is subject to liquidation by a privileged address which then writes off the debt on Moolah to suppliers. Even after a position has been liquidated, it is expected from users to clear their debt via the _repay control-flow, by repaying not only the remaining principal but also any interest or penalty.

Privileged Functions

- liquidate

Core Invariants:

INV 1: only addresses with the BOT role can be liquidated

INV 2: A position can only be liquidated once

Issue_02	Collateral is not locked after liquidation
Severity	**High**
Description	When a position is liquidated, the debt of the position is written off. Users are then unable to make any more borrowings until they clear their debt in that position due to the userNotPenalized modifier. However, there is nothing stopping the users from simply withdrawing their collateral tokens from the Morpho market, either via withdrawCollateral or by directly calling withdraw on the Morpho market. This collateral token can then be supplied to a different market to take out more loans, without clearing out the liquidated debt position first. Thus, liquidation doesn't actually lock the collateral of the user, allowing them to re-use it if possible. It has to be noted that this can be solved when making sure that the new scores are already set, such that a liquidated user will withdraw his collateral and it gets burned immediately.
Recommendations	Make sure the credit score is decreased before the liquidation call. Or disallow withdrawals while there are bad debt positions.
Comments / Resolution	Fixed by disallowing withdrawals when any position of the user isBadDebt. But it must be noted that this changes the behaviour of the contract and **disallows collateral withdrawal even when the position is healthy if a single loan goes into bad debt**.

Issue_03	Repayment of liquidated positions can be sandwich-attacked
Severity	**Medium**
Description	Whenever interest/penalty is repaid, these funds are flowing into the MoolahVault and will benefit depositors. This issue is now amplified because the full repayment amount will also flow into the MoolahVault (for liquidated positions). This can be trivially JIT-attacked by depositing into the vault -> repaying -> withdrawing from the vault.
Recommendations	Consider thinking about a withdrawal delay on the vault side. It would also require making shares not transferable.
Comments / Resolution	Acknowledged

Issue_04	Accumulation of zombie positions due to non-repayment of liquidated positions
Severity	**Low**
Description	Liquidated positions may accumulate because users are not required to repay them after liquidation. A user could leave a liquidated position untouched until: - period = 0 and penaltyRate = 0 - The grace period is increased. Under these conditions, the user may be able to borrow again using the same collateral. Additionally, only the user can withdraw the collateral from the market, which may result in inactive ("zombie") positions persisting indefinitely. It has to be noted that this only remains possible if the debt score has not yet been updated + synced because of the noDebt modifier within _borrow.
Recommendations	Add some functionality to manipulate the liquidated positions in case
Comments / Resolution	Acknowledged with the comment "period and penaltyRate will be not set to zero." More borrows can happen even if both are non-zero, in case the grace period is increased

Issue_05	config change can make liquidated position non-penalized
Severity	**Informational**
Description	Currently, it is possible that the config changes (ie. period is extended). In this specific scenario it is possible that a position which has been liquidated can be partially repaid as it is not considered as penalized anymore and thus no enforcement of full repayment is happening. Most of the time, exploits happen due to arbitrary user inputs or users invoking functions which are not meant to be invoked by users, one can argue that a large user flexibility is a great seed for exploits. Therefore, at BailSec, we are of the opinion that codebases should never provide more user flexibility than necessary during the normal business logic.
Recommendations	Consider being cautious whenever the period is increased.
Comments / Resolution	Acknowledged

Issue_06	Theoretical underflow risk for borrowShares math
Severity	**Informational**
Description	The borrowShares variable has been introduced and is reduced whenever a repayment happens. In the current "no-interest-fashion", there does not seem to be a case where a user might accidentally repay more than borrowed and thus trigger an underflow in this arithmetic operation. However, the risk cannot be considered as fully eliminated in case of unexpected edge-cases or similar scenarios.
Recommendations	Consider using the min approach and setting position.borrowedShares if repaidShares becomes suddenly larger.
Comments / Resolution	Acknowledged

Issue_07	Storage corruption during incorrect contract upgrade
Severity	**Informational**
Description	Compared to the previous version, new variables have been introduced in the FixedLoanPosition struct. This exposes a risk of storage collisions due to the way how the struct is accessed in storage.
Recommendations	Consider making sure that the storage is sound and using proper tooling during upgrades to prevent such collisions.
Comments / Resolution	Acknowledged

Issue_08	Lack of incentives to repay liquidated positions
Severity	**Informational**
Description	Once a position has been liquidated, the borrower may lack sufficient economic incentive to repay the outstanding debt. Repayment would require covering the principal, accrued interest, and any applicable penalties. As a result, the borrower may rationally choose to retain the borrowed funds and leave the remaining bad debt to be absorbed by the market's liquidity providers. If the protocol lacks a mechanism to enforce repayment, this behavior could adversely affect suppliers and undermine the market's economic stability. It is expected that this is handled via zero'ing out a user's CreditToken entitlement forever.
Recommendations	Consider introducing stronger post-liquidation incentives or enforcement mechanisms to ensure debt resolution.
Comments / Resolution	Acknowledged

Moolah - Differential

The Moolah contract undergoes a differential audit based on the following diff:

https://www.diffchecker.com/uMn79OWK/

The following changes have been made:

- Implementation of liquidateBrokerPosition function which allows for offsetting a position's debt to asset suppliers

Appendix: Liquidation

A new liquidation path has been introduced which is solely meant to be called by the CreditBroker implementation and simply writes off the outstanding debt towards the suppliers by decreasing the supply exchange rate in a similar fashion as during bad debt application via the normal liquidation mechanism.

Privileged Functions

- liquidateBrokerPosition

Core Invariants:

INV 1: liquidateBrokerPosition can only be called by the broker

Issue_09	Suppliers in CreditBroker markets are only subject to slashing without incentives
Severity	**Medium**
Description	In credit-broker markets, suppliers are exposed to bad debt risk but receive no explicit incentives (e.g., yields, fees, or rewards) to compensate for it. As a result, external liquidity providers may lack sufficient economic motivation to participate in these markets. This could lead to limited liquidity, with the protocol itself being the primary or sole supplier. If the protocol relies on external suppliers for market depth and sustainability, this design may reduce participation and negatively impact market growth.
Recommendations	Consider introducing incentive mechanisms for external suppliers to properly compensate them for the associated risk. Alternatively, if the intended design assumes that the protocol will be the sole liquidity provider, this issue can be acknowledged.
Comments / Resolution	Acknowledged

Issue_10	Frontrun of liquidation to bypass loss socialization
Severity	**Low**
Description	Whenever a liquidation happens, suppliers can simply withdraw their collateral to avoid the loss socialization, which more so increases the damage borne by the other suppliers.
Recommendations	Consider acknowledging this issue.
Comments / Resolution	Acknowledged

Issue_11	Liquidation does not check for collateralization
Severity	**Informational**
Description	Currently, the only source of truth for a liquidation is the expiration of a position without an actual check whether a position is indeed above the LTV. While this is a design decision it can be considered somehow unconventional.
Recommendations	Consider acknowledging this note.
Comments / Resolution	Acknowledged

Issue_12	liquidateBrokerPosition can be called by any broker
Severity	**Informational**
Description	The function liquidateBrokerPosition was intended to be used exclusively within credit broker markets. However, it can currently be called by any broker with a valid broker ID. This behavior may be problematic for brokers that are not designed to handle credit tokens. Allowing non-credit brokers to invoke this function could lead to unintended behavior.
Recommendations	Restrict access to liquidateBrokerPosition so that only designated credit brokers are permitted to call it.
Comments / Resolution	Acknowledged